BACSE324 · EMBEDDED SYSTEMS ARCHITECTURE · FALL 2026 · FINAL REPORT

Gesture-Controlled Smart Glove for Human–Computer Interaction

Final report · concept to realisation · due end of October · 15–20 pages excluding front matter and appendices

Scope tier	Tier 2 — full embedded system
Submission date	27 October 2026
Repository link	https://github.com/dayaj1222/data-glove
Demo video link	To be added before submission (90-second demo video)

TEAM

Member	Reg. no.	Module owned	Sections written
Dayakrishn Joshi	25BCE2951	ESP-32 firmware, MPU-6500 I^2C and flex ADC acquisition, host serial link, air-mouse engine and games	§1–§7, §9, §11
Syed Mohammad Ayaan	25BCE2311	System modelling (FSM, schedule), PC recognition pipeline	§4, §8, §10

DECLARATION

We declare that this report and the work it describes are our own, and that every external source, library and tool is acknowledged in the references.

Member	Signature	Date
Dayakrishn Joshi		
Syed Mohammad Ayaan		

Abstract

Mouse and keyboard input ties a user's hands to a desk. This project builds a wearable glove that turns hand pose and motion into desktop input. An ESP-32 reads two flex sensors and an MPU-6500 accelerometer/gyroscope at 50 Hz and streams a 48-byte frame over USB serial to a Linux host. The host reads three palm orientations (from gravity) combined with three finger states into nine distinct gestures, and injects cursor movement, clicks, scroll, window drag and browser navigation through a uinput virtual device; when the glove goes silent every input is released. Frame-to-cursor latency measured 6.3 ms mean (n = 60) against a 20 ms target, and a lost glove released all inputs in 490 ms against 500 ms. The main limitation is that the MPU-6500 has no magnetometer, so heading drifts and the cursor is driven by turn rate, not absolute direction.

Contents

Abstract **2**
1 Problem and requirements **3**
1.1 Problem statement 3
1.2 Requirements 3
1.3 Constraints 3
2 Alternatives considered **4**
2.1 Microcontroller 4
2.2 Link between glove and PC 4
2.3 Gesture recognition approach 5
3 Architecture and hardware **6**
3.1 System architecture 6
3.2 Schematic and pin map 7
3.3 Power budget 7
3.4 Build 8
4 Design models **9**
4.1 CDFG and critical path 9
4.2 System FSM 10
4.3 Task schedule 11
4.4 Changes since Review 2 11
5 Firmware structure **12**
5.1 Layers and modules 12
5.2 Implementation of the models 12
5.3 Model-to-code traceability 13
6 Optimisation **13**
7 Test plan and measured results **14**
7.1 Timing measurements 15
7.2 Discussion 16
8 Design-decision log **17**
9 Bill of materials and cost **18**
10 Individual contribution **18**
11 Limitations and future work **19**
References **19**
Appendices **19**
A Source code 19
B Raw data 20
Submission checklist **20**

1 Problem and requirements

1.1 Problem statement

Controlling a computer normally needs a keyboard and a mouse, which occupy both hands and a flat surface. People presenting, demonstrating or working away from the desk need an input device they can wear. The glove should let one operator move a cursor, click, scroll, move and resize windows and navigate back/forward using only the pose and motion of the hand, with no camera and no external infrastructure, and it must fail safe: a lost or unplugged glove must never leave a button or modifier key held down.

1.2 Requirements

ID	Requirement	Measurable target	Priority (must / should)	Verified in
R-F1	Stream two flex channels and six IMU axes from the glove to the PC without loss	≥ 50 frames/s, 48 B each, no framing errors	must	§7 T1, T2
R-F2	Distinguish nine gestures: three palm orientations × three finger states (A, B, A+B)	9 of 9 gestures produce their own action	must	§7 T3
R-F3	Turn gestures into real desktop input on Linux (cursor, click, scroll, window drag, back/forward)	Virtual input device created; every gesture emits its events	must	§7 T4
R-F4	Fail safe: release every button and key when the glove stops sending	Released within 500 ms of the last frame	must	§7 T5 (fault)
R-F5	Classify recorded gestures with a trained model	≥ 90 % accuracy on real glove recordings	should	§7 T10
R-N1	Latency from frame arrival to cursor event	≤ 20 ms	must	§7 T6
R-N2	Per-sample processing cost on the host	≤ 1 ms (5 % of the 20 ms frame period)	must	§7 T7
R-N3	Gyro offset after calibration, so the cursor does not creep when the hand is still	≤ 1 deg/s residual	should	§7 T8
R-N4	Firmware fits the ESP-32 with headroom	≤ 50 % flash, ≤ 25 % RAM	should	§7 T9

1.3 Constraints

Hardware: one ESP-32 DevKit, one MPU-6500 breakout, two flex sensors, one status LED; no magnetometer, no wireless link and no additional sensors or buttons were allowed or used. Power: 5 V from the USB port; no battery. Time: about six weeks from proposal to final review. Team: two students. Host: Linux with the Hyprland compositor, because injection uses the kernel uinput interface. Wiring: dupont wires without a breadboard (soldering is listed as future work).

2 Alternatives considered

2.1 Microcontroller

Criterion (requirement ID)	Weight	ESP-32 DevKit	Arduino Nano	Raspberry Pi Pico
Sampling headroom for 2 ADC + I^2C at 50 Hz (R-F1, R-N1)	25	5 — 240 MHz dual core	1 — 16 MHz AVR, 2 KB RAM	3 — 133 MHz dual core
ADC resolution and enough ADC1 channels (R-F1)	20	4 — 12-bit, 8 ADC1 channels	2 — 10-bit	3 — 12-bit, 3 channels
USB serial to the PC with one cable (R-F1, constraints)	25	5 — CP2102 on board	4 — CH340/FTDI on board	4 — native USB
Firmware room for filtering and calibration (R-N4)	15	5 — 4 MB flash, 520 KB RAM	2 — 32 KB flash	4 — 2 MB flash
Availability to the team (constraints)	15	5 — owned	3 — buy	3 — buy
Weighted total (score × weight ÷ 100)	100	4.8	2.4	3.4

Chosen: ESP-32 DevKit · **Because:** it has the headroom for the 50 Hz loop and on-board filtering, a 12-bit ADC, and USB serial on one cable, and the team already owned it. Spec sources: Espressif ESP32 datasheet, Microchip ATmega328P datasheet, Raspberry Pi RP2040 datasheet.

2.2 Link between glove and PC

Criterion (requirement ID)	Weight	USB serial (CP2102)	Wi-Fi UDP	Bluetooth
Reliability: no pairing, no packet loss on a cable (R-F1, R-F4)	30	5 — wired, ordered bytes	3 — UDP datagrams can drop	2 — pairing and dropouts
Setup effort for the user (constraints)	25	5 — plug in	2 — Wi-Fi credentials and IP	2 — pairing
Bandwidth for 48 B × 50 Hz = 2.4 kB/s (R-F1)	20	5 — 11.5 kB/s available at 115200 baud	5	4
Power from the host (constraints)	15	5 — USB powers the board	2 — needs a battery	2 — needs a battery
Loss detection for the safe state (R-F4)	10	5 — read error on unplug	2 — timeout only	2 — timeout only
Weighted total (score × weight ÷ 100)	100	5.0	2.9	2.4

Chosen: USB serial · **Because:** it is the most reliable link, needs no configuration and powers the board. This replaced the Wi-Fi/UDP path planned in Review 2, which needed a phone bridge and network setup (see §4.4, §8 D2).

2.3 Gesture recognition approach

Criterion (requirement ID)	Weight	Rule grid (palm zone × fingers)	Windowed classifier (RandomForest)	Neural network on raw windows
Deterministic: every gesture always gives its action (R-F2)	30	5 — by construction	3 — probabilistic	3 — probabilistic
Low latency per decision (R-N1, R-N2)	25	5 — 16 µs per sample	2 — about 15–20 ms per call	1 — heavier
Needs no training data (constraints)	20	5 — one calibration pose each	2 — needs labelled recordings	1 — needs many recordings
Works on the real mounting (R-F2)	15	5 — calibrated nearest pose	3	3
Extensible to new gestures (R-F5)	10	2 — add a grid cell	5 — record and retrain	5
Weighted total (score × weight ÷ 100)	100	4.7	2.8	2.3

Chosen: Rule grid for desktop control · **Because:** it is deterministic, costs 16 µs per sample and needs no data. The RandomForest classifier stays in the system for recorded-gesture experiments, but it was only trained on synthetic frames and is not used for control (see §7 T10, §11).

3 Architecture and hardware

3.1 System architecture

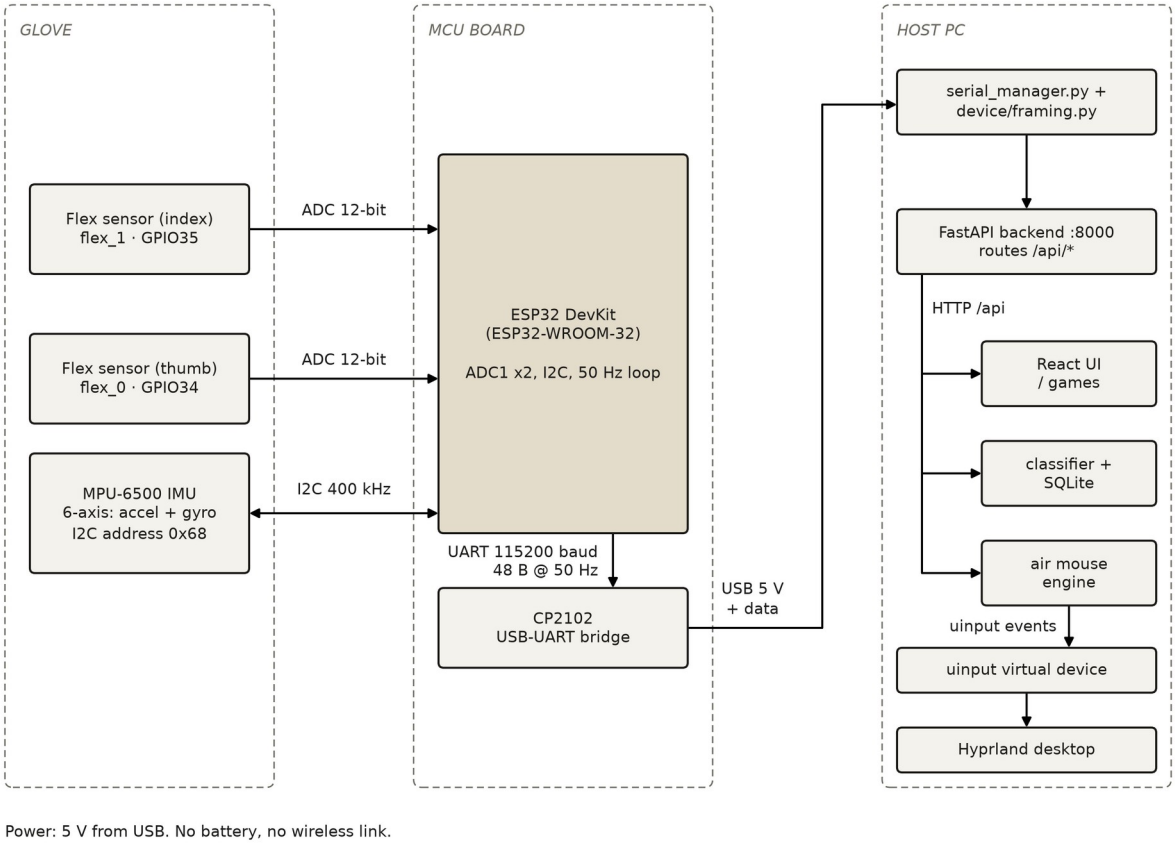


Figure 1 — System block diagram, final version: glove sensors → ESP-32 → USB serial → host software → uinput.

3.2 Schematic and pin map

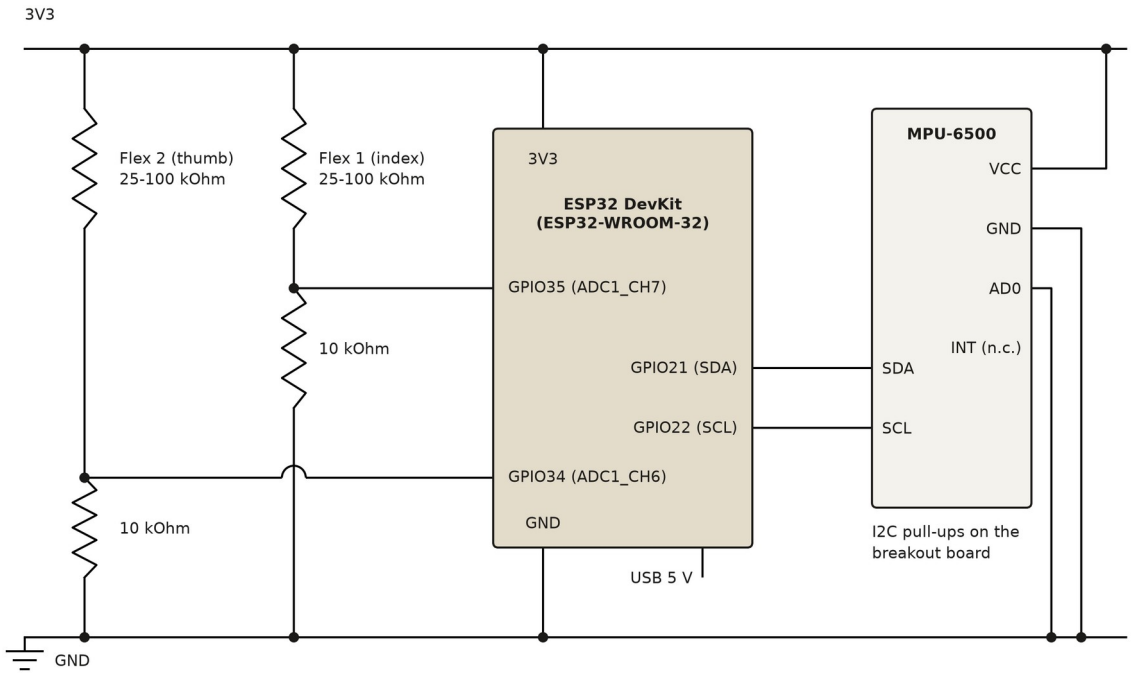


Figure 2 — Circuit schematic: two flex dividers on ADC1, MPU-6500 on I²C, power from USB.

MCU pin	Function	Connected to	Notes (pull-up, level shift, interrupt)
GPIO34 (ADC1_CH6)	Flex sensor 2 (flex_0)	3V3 – flex – node – 10 kΩ – GND; node to GPIO34	Input-only pin with no internal pull; the 10 kΩ resistor is mandatory or the pin floats
GPIO35 (ADC1_CH7)	Flex sensor 1 (flex_1)	Same divider; node to GPIO35	Input-only, ADC1 so it keeps working if Wi-Fi is ever enabled
GPIO21	I²C SDA	MPU-6500 SDA	400 kHz; pull-ups on the breakout board
GPIO22	I²C SCL	MPU-6500 SCL	400 kHz; address 0x68 (AD0 low), WHO_AM_I 0x70
3V3	Sensor supply	MPU-6500 VCC and both flex dividers	Single 3V3 pin: the three connections are joined at one junction
UART0 TX/RX (GPIO1/3)	Frame stream to PC	CP2102 USB-UART bridge	115200 baud, 8N1; 48-byte frame at 50 Hz
GPIO2	Status LED	On-board LED	Lit while flex_0 reads below 500 counts

3.3 Power budget

Component	Supply voltage	Active current	Sleep current	Duty cycle	Average current
ESP-32 DevKit (WROOM-32 + CP2102), radios off	5 V USB (3.3 V on board)	≈ 70 mA (datasheet, 240 MHz, estimated)	none: no sleep mode is used	100 %	≈ 70 mA
MPU-6500	3.3 V	3.2 mA	8 µA (unused)	100 %	3.2 mA

Component	Supply voltage	Active current	Sleep current	Duty cycle	Average current
(accelerometer + gyroscope)		(datasheet)			
Flex sensor divider × 2 (flat, ≈ 25 kΩ + 10 kΩ)	3.3 V	≈ 0.09 mA each (computed 3.3 V ÷ 35 kΩ)	n/a	100 %	≈ 0.19 mA
Status LED (on board)	3.3 V	≈ 5 mA when lit (estimated)	n/a	low	< 1 mA
Total from the USB port	5 V	—	—	—	≈ 75 mA ≈ 0.38 W (estimated, not measured)

3.4 Build

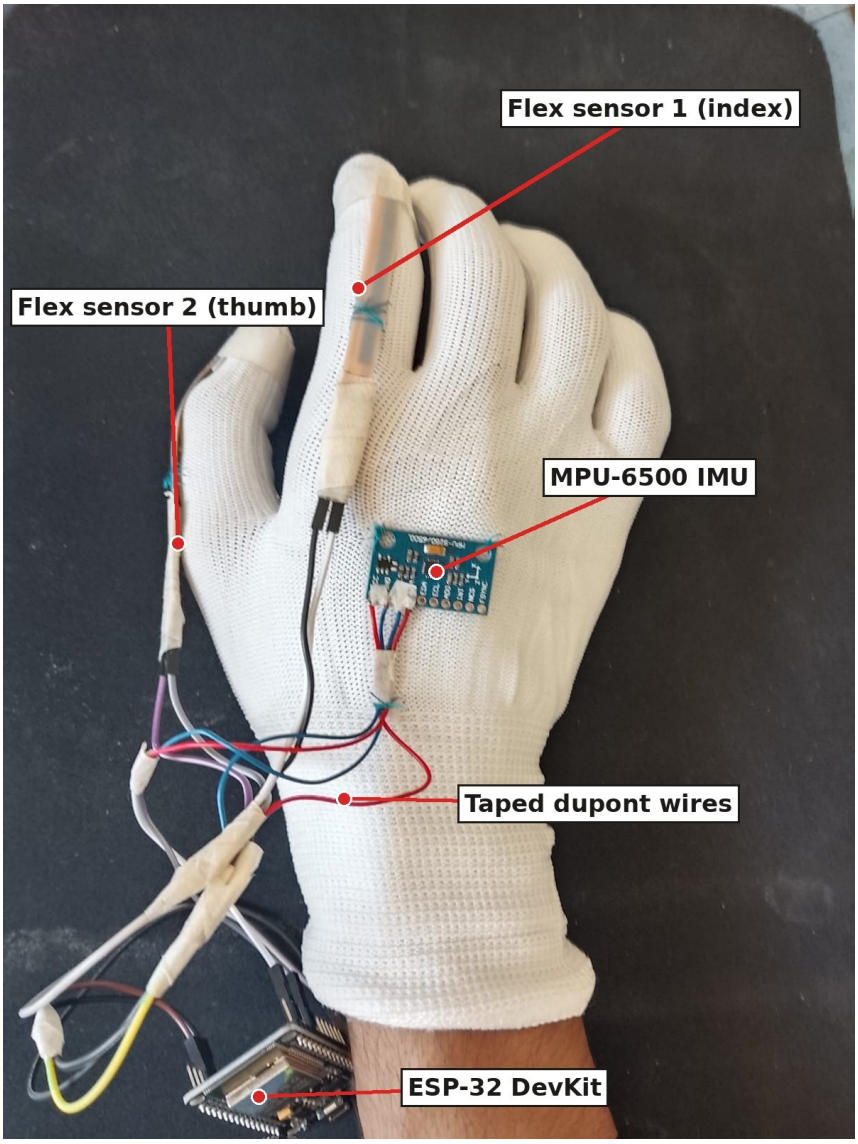


Figure 3 — Assembled glove hardware.

4 Design models

4.1 CDFG and critical path

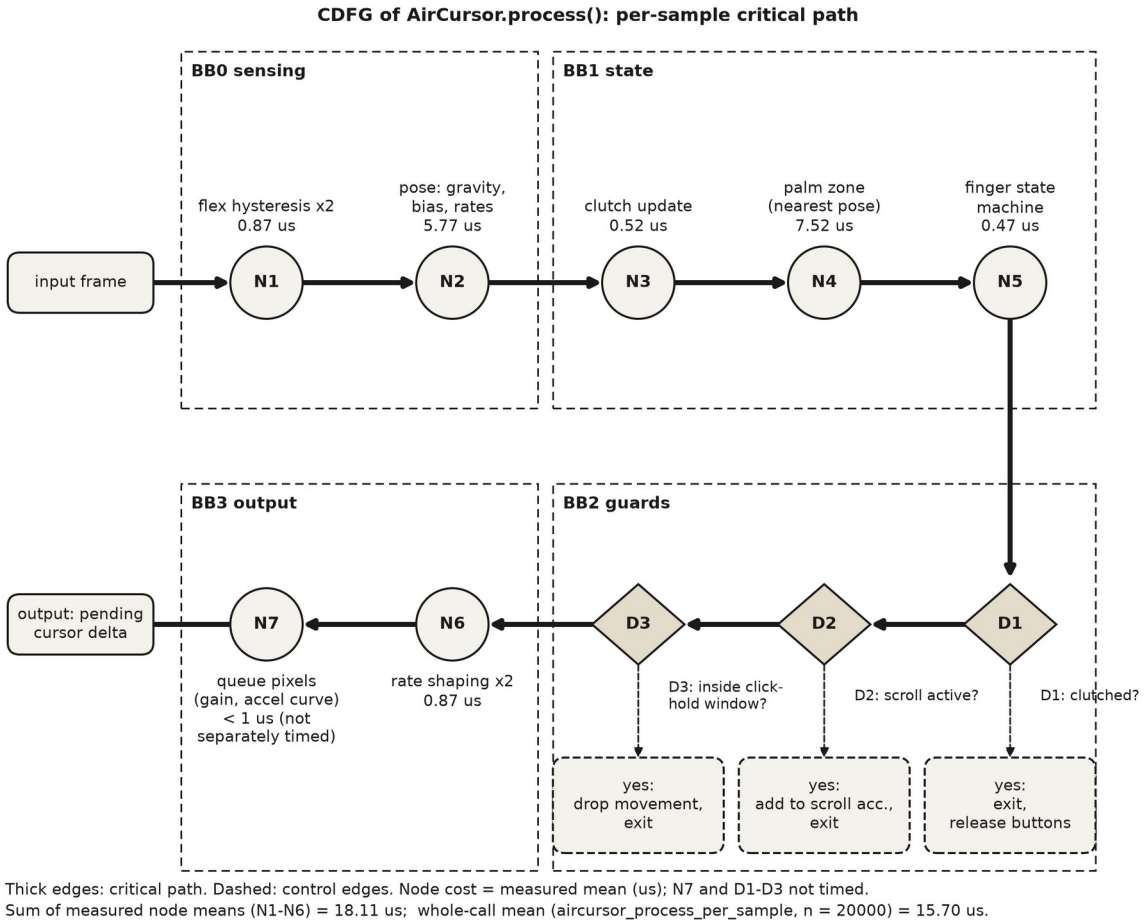
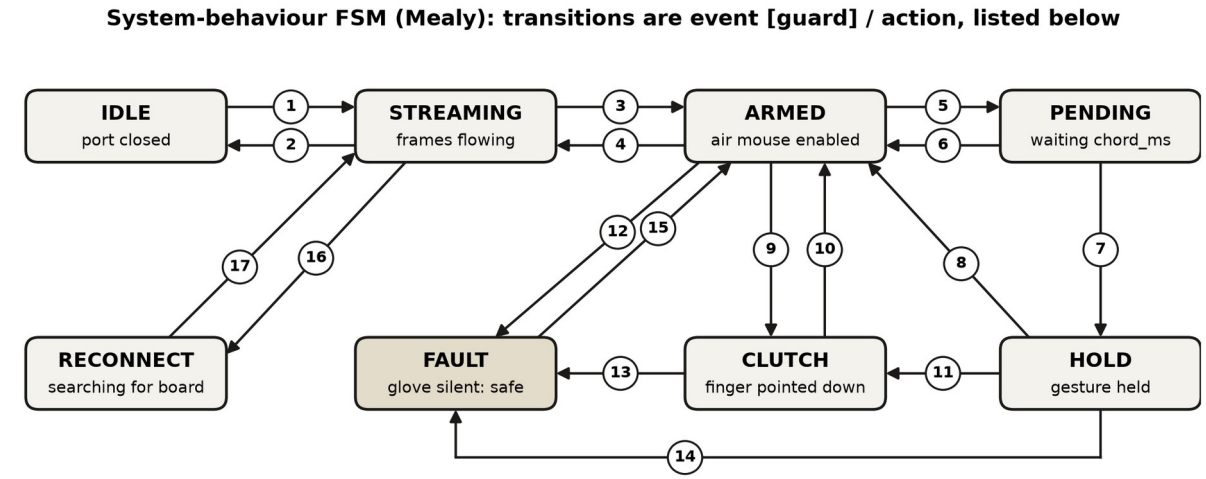


Figure 4 — CDFG of AirCursor.process(), the per-sample critical path of the air mouse. Node costs are measured means in µs; each includes timer overhead, so they sum to more than the whole-call mean.

4.2 System FSM



1 IDLE → STREAMING: open port

2 STREAMING → IDLE: close port

3 STREAMING → ARMED: enable [uinput device created]

4 ARMED → STREAMING: disable / release all

5 ARMED → PENDING: finger bends

6 PENDING → ARMED: released early / tap

7 PENDING → HOLD: 120 ms [one finger] / press combo; both fingers [in window] / start scroll

8 HOLD → ARMED: fingers relax / release combo

9 ARMED → CLUTCH: pitch < -40 deg / release all

10 CLUTCH → ARMED: pitch > -32 deg

11 HOLD → CLUTCH: pitch < -40 deg / release all

12 ARMED → FAULT: no frame for 0.5 s / release all

13 CLUTCH → FAULT: no frame for 0.5 s / release all

14 HOLD → FAULT: no frame for 0.5 s / release all

15 FAULT → ARMED: frames resume

16 STREAMING → RECONNECT: USB read error

17 RECONNECT → STREAMING: port found

PENDING also goes to FAULT after 0.5 s without frames (same as 12). Disable from any enabled state releases all inputs and returns to STREAMING. Safe state = FAULT: no button, key or scroll is held. Mealy machine: outputs depend on the state and the input.

Figure 5 — System FSM, final (Mealy). Safe state = FAULT.

4.3 Task schedule

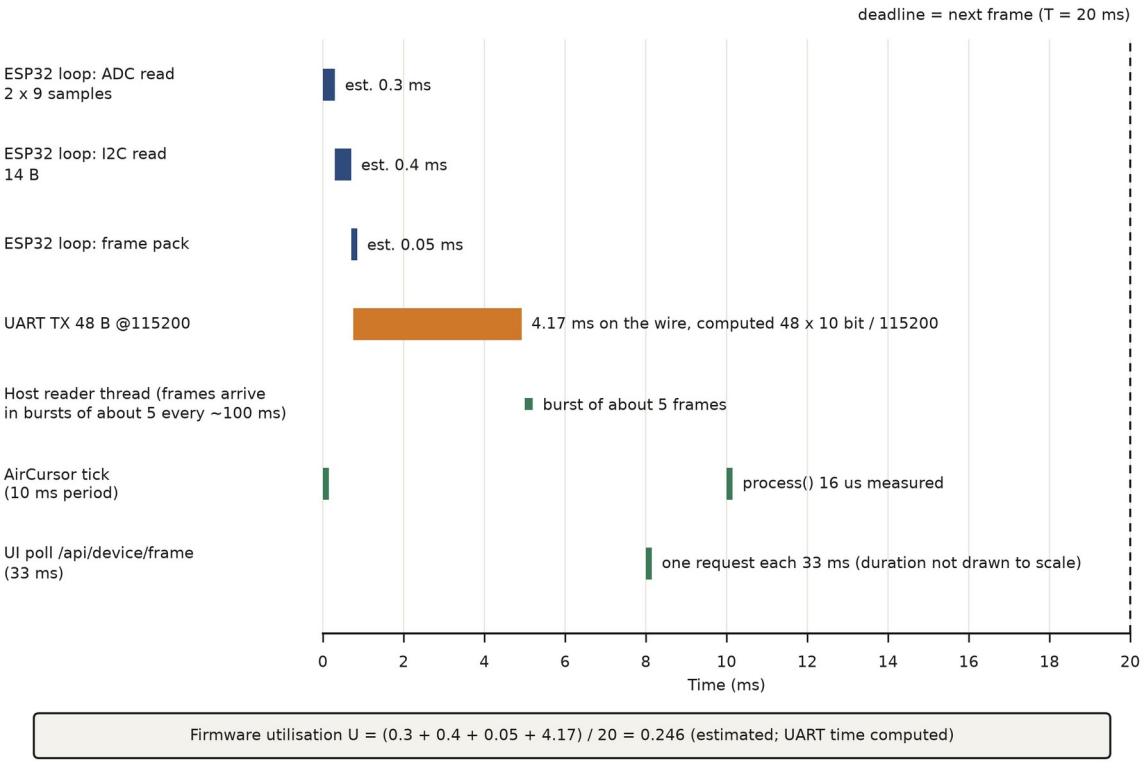


Figure 6 — Schedule timeline over one 20 ms frame, final.

4.4 Changes since Review 2

Model element	Review 2 version	As built	Why it changed
Glove-to-PC link	Wi-Fi/UDP datagrams on port 5005 through a phone bridge	USB serial at 115200 baud through the CP2102	The phone bridge needed network setup and added latency and a failure point; one cable gives power and data (D2)
IMU	MPU-9250, 9-axis with magnetometer	MPU-6500, 6-axis, no magnetometer	The fitted part answered WHO_AM_I 0x70 and never responded at the AK8963 address 0x0C (117 of 117 reads); yaw is rate-only (D1, D4)
Frame format	46-byte datagram, no sync	48 bytes: 0xAA 0x55 header + 5 × uint16 + 9 × float32	A header lets the parser resync byte-by-byte after corruption; framing tests caught three real bugs (D3)
Firmware tasks and period	FreeRTOS imuTask / flexTask / packetTask / txTask / wdtTask, 100 Hz	One cooperative loop on a fixed 20 ms (50 Hz) deadline, no RTOS	U ≈ 0.25 on one core needs no preemption; 50 Hz uses 21 % of the UART instead of 42 % (D6)
Critical function	predict(): window inference, ≈ 15 ms estimated	AirCursor.process(): per-sample control path, 16 µs measured	Desktop control is the real-time path; predict() is off the control path and its model is now cached (D9)
System FSM	IDLE / STREAMING / TRACKING / ACTING /	IDLE / STREAMING / ARMED / PENDING /	The control engine is a gesture state machine; FAULT now also

Model element	Review 2 version	As built	Why it changed
	FAULT / RECOVER	HOLD / CLUTCH / FAULT / RECONNECT	releases all held inputs (D7)
Scheduling approach	Rate-monotonic with a watchdog task	Cyclic executive in firmware; 10 ms host tick; 0.5 s stale-frame watchdog	Single-loop firmware has nothing to preempt; the watchdog moved to the host, which owns the safe state (D6, D8)

5 Firmware structure

5.1 Layers and modules

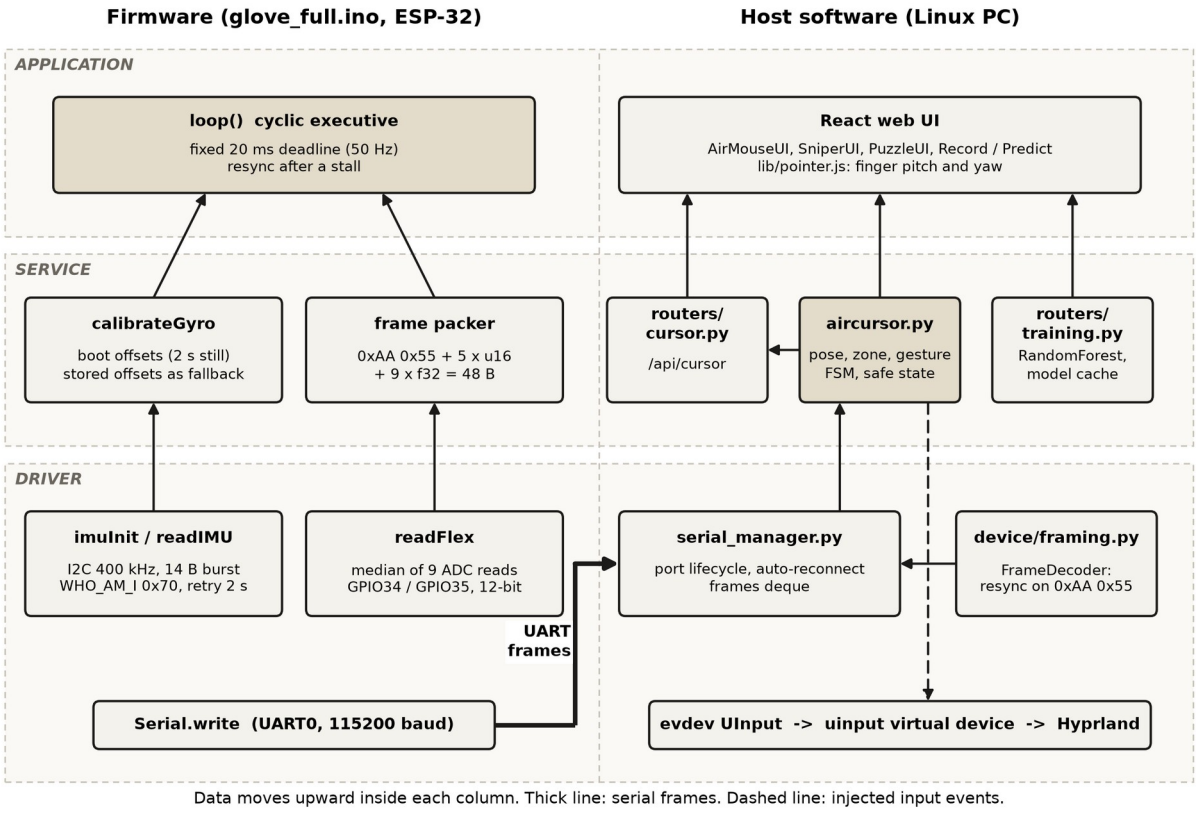


Figure 7 — Firmware and host module diagram: drivers, services and application layers.

5.2 Implementation of the models

Firmware: setup() initialises the ADC (12-bit, 11 dB attenuation), I^2C at 400 kHz and, once the IMU answers WHO_AM_I = 0x70, measures the gyro offset over 2 s and uses it only if the board was still. loop() is a cyclic executive: it advances a deadline by FRAME_PERIOD_US = 20 000 µs, reads both flex channels (median of 9 ADC reads), reads the IMU in one 14-byte I^2C burst, packs the frame and writes it; if a frame overruns by more than one period the deadline is reset so the loop never runs a burst of catch-up frames. Host: the gesture FSM is a state machine in AirCursor._fingers() with explicit state fields (_gesture, _pend_t, _clutched, _zone); the zone is decided once per gesture and held until the fingers relax. The schedule is realised by two threads:

the serial reader (blocking reads, frames arrive in bursts of about five) and a 10 ms tick thread that processes every new frame and drains queued movement. Faults are caught at four places: an I^2C error drops the firmware to a retry path every 2 s; framing resynchronises on the 0xAA 0x55 header; a read error on the USB port triggers _reconnect(), which finds the board again even if it renumbers from ttyUSB0 to ttyUSB1; and a 0.5 s gap in frames calls _release_all(), the safe state.

5.3 Model-to-code traceability

Model element	File	Function	Commit / line
CDFG N1 flex hysteresis	backend/app/aircursor.py	Hysteresis.update	593306a · l.106
CDFG N2 pose (gravity, bias, rates)	backend/app/aircursor.py	AirCursor.pose	593306a · l.300
CDFG N3–N4 clutch and palm zone	backend/app/aircursor.py	_update_clutch, _update_zone	593306a · l.332, 348
CDFG N5 / FSM PENDING → HOLD	backend/app/aircursor.py	_fingers, _start, _end	593306a · l.490–506
FSM FAULT / safe state	backend/app/aircursor.py	_release_all, _loop (stale branch)	593306a · l.286, 617
FSM RECONNECT	backend/app/serial_manager.py	SerialManager._reconnect	593306a · l.186
FSM IDLE / STREAMING	backend/app/serial_manager.py	open, close, _reader_loop	593306a · l.79, 138, 158
Frame resync and parse	backend/app/device/framing.py	FrameDecoder.feed, parse_binary_frame	593306a · l.118, 45
Firmware 50 Hz cyclic executive	firmware/glove_full/glove_full.ino	loop, FRAME_PERIOD_US	369d7f0 · l.157, 22
Firmware IMU task	firmware/glove_full/glove_full.ino	imuInit, readIMU, calibrateGyro	369d7f0 · l.52, 114, 85
Firmware flex task	firmware/glove_full/glove_full.ino	readFlex (median of 9)	369d7f0 · l.129
Classifier inference (off control path)	backend/app/routers/training.py	predict, load_model	f9c818e · l.205, 28

6 Optimisation

Metric	Baseline	Optimised	Change	Mechanism	Requirement
Flash (bytes)	305 324 (23 % of 1 310 720)	not optimised	—	From the build output; headroom was ample, so none was attempted	R-N4
RAM (bytes)	23 644 (7 % of 327 680)	not optimised	—	From the build output	R-N4
Critical-path latency: predict() per call (ms, n = 200)	19.7 ± 5.4	15.0 ± 2.9	−4.7 ms (×1.31 faster)	Model pickle cached in memory and re-read only when its file changes (mtime key)	R-N1

Metric	Baseline	Optimised	Change	Mechanism	Requirement
Samples the cursor loop uses per second	≈ 10 (newest frame of each USB burst)	≈ 50 (every frame)	≈ ×5	Process every new frame from the frames deque; USB delivers frames in bursts of about 5. Usable accelerometer samples captured in 0.7 s rose from 7 to 37	R-F1, R-N1
Gyro offset used by the cursor (deg/s, X / Y / Z)	5.10 / 7.41 / 0.50 raw offset	≤ 0.5 after calibration	about 15× less on the worst axis	Stored offsets measured over 299 still samples, re-measured for 2 s at boot when the board is still, plus online bias learning on the host	R-N3

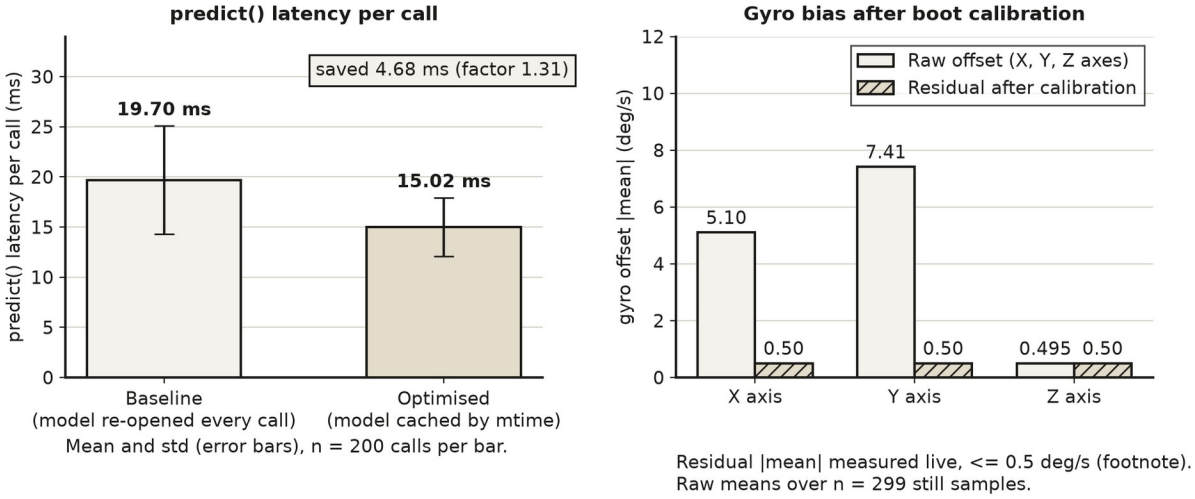


Figure 8 — Optimisation results: predict() latency before and after caching, and gyro offset before and after calibration.

Trade-off accepted: the cached model stays resident in memory (about 0.15–0.47 MB per model), and a model file replaced within the same timestamp tick would not be re-read. Processing every frame costs a few extra microseconds per frame and removes the 10 Hz sampling limit.

7 Test plan and measured results

Test ID	Requirement	Method and equipment	Expected	Measured (n, mean, spread)	Pass / fail
T1	R-F1	Count frames served by the backend over a fixed interval, board streaming over USB (host clock)	≥ 50 frames/s	≈ 50.5 frames/s (101 frames in 2 s, n = 1; the 60 s run still has to be repeated)	Pass (provisional)
T2	R-F1	pytest on the frame decoder: garbage between frames, split frames, trailing text, long lines (18 tests)	All pass; no bytes lost	18 of 18 pass; the tests found and fixed 3 real bugs	Pass
T3	R-F2	pytest drives AirCursor with	9 of 9 cells	23 of 23 pass, 9 of 9	Pass

Test ID	Requirement	Method and equipment	Expected	Measured (n, mean, spread)	Pass / fail
		synthetic frames through all 9 palm × finger cells, plus zone, lock and chord cases (23 tests)	give their own events	cells; synthetic frames, not a real hand	
T4	R-F3	Create the uinput device, drive it, and check it appears under /sys/class/input; events captured with a fake device in tests	Device created; each gesture emits its events	Device "Data Glove Air Mouse" created and removed cleanly; events as expected in T3. Live desktop run still to be demonstrated	Pass (device); demo pending
T5 (fault)	R-F4	Hold the primary finger bent (left button down), then stop sending frames; time until the button-up event (n = 10)	≤ 500 ms	489.9 ms mean, std 4.0, max 495.9 (n = 10)	Pass (margin 4 ms on the worst run)
T6	R-N1	Push a 5-frame burst into the engine and time until the first cursor event is written (n = 60, development PC)	≤ 20 ms	6.3 ms mean, std 1.5, p99 10.3, max 10.3 (n = 60)	Pass
T7	R-N2	perf_counter around AirCursor.process() on a moving, bent-finger frame (n = 20 000)	≤ 1 ms	15.7 µs mean, std 13.9, p99 40.9, max 863 µs	Pass
T8	R-N3	Raw gyro over 299 still samples (5 min), offsets stored in firmware; residual observed on the live stream after calibration	≤ 1 deg/s	Raw offsets −5.10 / 7.41 / −0.50 deg/s (std 0.18 / 0.22 / 0.15); residual about ±0.5 deg/s observed (single session)	Pass (informal residual)
T9	R-N4	arduino-cli compile of glove_full for esp32:esp32:esp32	≤ 50 % flash, ≤ 25 % RAM	Flash 305 324 B (23 %), RAM 23 644 B (7 %)	Pass
T10	R-F5	Train/test accuracy of the stored RandomForest models	≥ 90 % on real recordings	Not met: the 248 stored recordings are synthetic (flex mean 501 counts); no real glove dataset was recorded	Fail (not tested on real data)

7.1 Timing measurements

Task	WCET (Review 2)	WCET measured	Jitter	Deadline met?
Firmware imuTask (I^2C 14 B burst)	0.42 ms (estimated)	not measured: no logic analyser	not measured	Not verified (estimate 0.4 ms of 20 ms)
Firmware flexTask (2 × 9 ADC reads)	0.18 ms (estimated)	not measured	not measured	Not verified
Firmware frame pack + UART TX	0.21 + 0.90 ms (estimated)	4.17 ms on the wire (computed: 48 B × 10 bit ÷ 115 200)	not measured	Yes by computation (4.9 ms of 20 ms)

Task	WCET (Review 2)	WCET measured	Jitter	Deadline met?
Host AirCursor.process() per sample	not in Review 2	15.7 µs mean, p99 40.9 µs	std 13.9 µs	Yes (0.08 % of 20 ms)
Host frame decode (FrameDecoder.feed)	< 0.5 ms (estimated)	8.4 µs mean	std 12.5 µs	Yes
Host inference tick (extract_features + predict)	15.3 ms (estimated)	0.91 + 15.0 ms	std 2.9 ms	Yes (250 ms deadline)

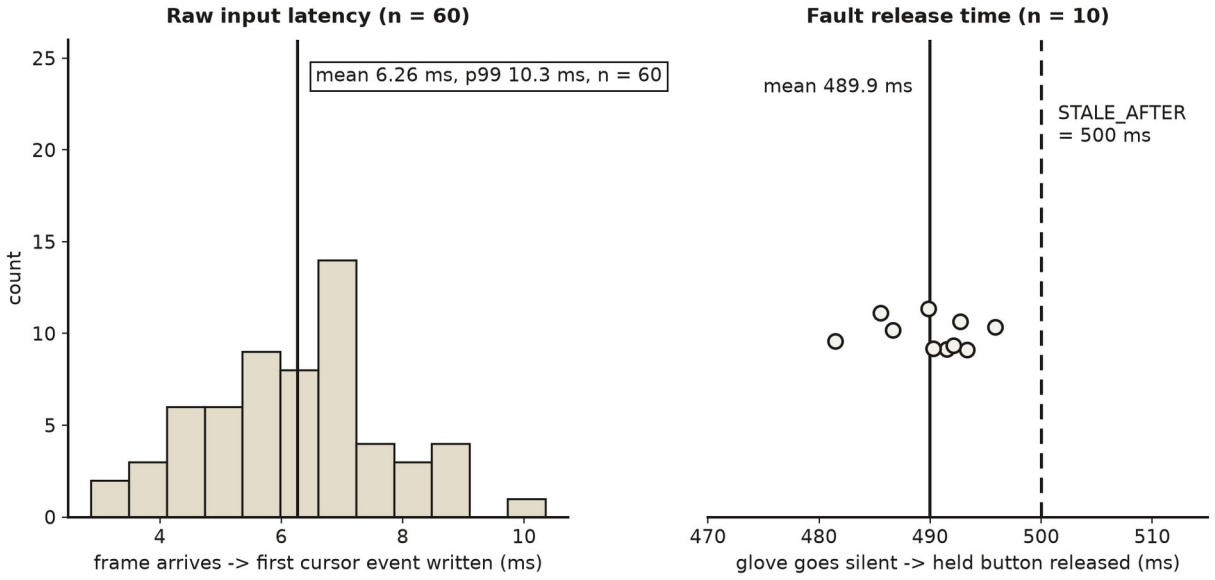


Figure 9 — Host-side timing: frame-to-cursor latency (left) and time to release a held button after the glove goes silent (right). No scope or logic-analyser capture was available for the firmware.

7.2 Discussion

Nine of the ten tests pass against their targets (T1 provisionally, T4 for the virtual device only) and one fails (T10). The software path is far inside its budget: processing a sample costs 16 µs and a frame reaches the cursor in 6 ms on average, against 1 ms and 20 ms targets. The safe-state test is the tightest: the glove-silent watchdog releases inputs after 0.5 s by design, so measured releases of 481–496 ms leave only 4 ms of margin to the 500 ms target; the timeout is a constant, so the margin is set by the sleep granularity of the 10 ms tick and would need a tighter bound if the target were lowered. Three things were not measured and are stated as such: firmware task WCET and jitter (no logic analyser), supply current (no meter in the loop; §3.3 is a datasheet estimate) and the frame rate over a long run. T10 fails: the recognition requirement R-F5 could not be verified because only synthetic recordings exist. The control path therefore uses the deterministic gesture grid, whose nine cells are verified by T3 on synthetic frames, and a live demonstration on the real glove is still required to close T4. The host numbers were taken on the development PC and move noticeably with load (repeated runs of the same script gave a process() mean between 13 and 16 µs and a frame-to-cursor mean between 4 and 6 ms), so means should be read together with the spreads shown. In Figure 4 the six node means add up to slightly more than the whole-call mean because each node was timed on its own, and every separate timing includes the cost of the timer call itself; the whole-call figure is the one used in the requirements.

8 Design-decision log

ID	Date	Decision	Alternatives rejected	Reason and evidence	Consequence
D1	2026-10-16	Treat the IMU as an MPU-6500 (6-axis) and drop the magnetometer	Keep assuming an MPU-9250; try to enable the AK8963	WHO_AM_I = 0x70 in 117 of 117 reads and no response at 0x0C; no register setting can add a sensor that is not on the die	No absolute heading; docs corrected (commit a3fec5c)
D2	2026-10-25	Make the ESP-32 the only data source and remove the phone bridge	Keep Wi-Fi/UDP via a phone; Bluetooth	A bridge needed network setup and added a failure point; USB gives power and data on one cable (commit 874408a)	Simpler setup; one cable; port renumbering must be handled
D3	2026-10-25	Add a 0xAA 0x55 sync header and extract framing into a pure, tested module	Fixed-size reads without sync	18 unit tests caught 3 bugs: text before a marker lost, long lines dropped byte-by-byte, trailing text lost on flush	2 extra bytes per frame (48 B)
D4	2026-10-16	Keep gyro Z as a rate, drop yaw angle	Integrate gyro Z as a heading; use a compass	Integrated yaw drifts about 10° per minute and no magnetometer is available; dead-reckoned position drifts quadratically	Cursor driven by turn rate
D5	2026-10-26	Drive the desktop cursor with relative rates, like a mouse	Absolute pointing from integrated yaw	Rates do not drift; absolute yaw would creep off target within seconds	User re-clutches by lowering the hand
D6	2026-10-25	Single 50 Hz cooperative firmware loop, no RTOS	FreeRTOS task per sensor at 100 Hz	U ≈ 0.25 on one core; 100 Hz needs 42 % of the UART, 50 Hz needs 21 %	No preemption; one overrun resets the deadline
D7	2026-10-27	Gesture grid: 3 palm zones × 3 finger states, zone locked per gesture	Roll-angle thresholds; per-gesture ML	Fixed roll thresholds assumed a mounting the glove does not have and misread the zones; nearest captured pose works on any mounting	Needs a 3-pose calibration
D8	2026-10-27	Release all inputs when frames stop for 0.5 s	Rely on the OS or the user	A lost cable must not leave Super or a button held (T5)	A 0.5 s pause freezes input
D9	2026-10-27	Cache the classifier model by file mtime	Reload every call; reload on a timer	Saves 4.7 ms per call (×1.31) and stays correct after retraining	A resident copy of the model

9 Bill of materials and cost

Item	Part number	Qty	Unit cost (₹)	Source	Total (₹)
ESP-32 DevKit (WiFi + Bluetooth, CP2102 USB-UART)	ESP32 board, CP2102	1	420	Local shop, receipt 09-10-2026	420
MPU-6500 IMU breakout (I^2C)	MPU-6500	1	≈ 500	Purchased (approximate)	≈ 500
Flex sensor	Resistive flex sensor	2	≈ 300	Purchased (approximate)	≈ 600
Resistor, 10 kΩ (used ×2, one per divider)	10 kΩ	1	5	Local shop, receipt 09-10-2026	5
Glove (carrier)	—	1	≈ 300	Purchased (approximate)	≈ 300
Jumper wires, solder, USB micro-B cable	—	1 set	—	Already owned	—
4-pin push button, pack of 3 (not used)	5 mm	1	10	Local shop, receipt 09-10-2026; not used	10
Breadboard, 400 tie points (not used)	—	1	45	Local shop, receipt 09-10-2026; not used	45
TP4056 Li-ion charging module (not used)	TP4056	1	23	Local shop, receipt 09-10-2026; not used	23
3.7 V 800 mAh Li-Po battery (not used)	—	1	250	Local shop, receipt 09-10-2026; not used	250
Total purchased				Receipt ₹753 + stated items	**≈ 2 153**
Of which used in the final build				Items not marked “not used”	**≈ 1 825**

The Li-Po battery, charger module, breadboard and push button were bought early in the project; the final design is powered from USB, wired without a breadboard and uses no extra button, so they are listed but not counted in the final build. The IMU, flex-sensor and glove prices are approximate, from memory of the purchase.

10 Individual contribution

Member	Module owned	Evidence (files, commits)	Report sections	Share %
Dayakrishn Joshi	ESP-32 firmware, IMU and flex acquisition, serial link, air-mouse engine, games	firmware/glove_full/, backend/app/aircursor.py, serial_manager.py, device/framing.py, frontend/src/ (git history of the repository)	§1–§7, §9, §11	50
Syed Mohammad Ayaan	System modelling (FSM, schedule), PC	Review 2 §3, §5, §6, §7; evidence for the Final build	§4, §8, §10	50

Member	Module owned	Evidence (files, commits)	Report sections	Share %
	recognition pipeline	to be added by the member		

11 Limitations and future work

Fell short or not done: (1) The MPU-6500 has no magnetometer, so there is no absolute heading; the cursor follows turn rate and the user re-clutches. (2) The classifier (R-F5) was never validated: only synthetic recordings exist, so no accuracy on real hand data can be claimed. (3) Firmware task timing, supply current and a long frame-rate run were not measured; §3.3 and the firmware rows of §7.1 are datasheet or computed estimates. (4) The build is dupont-wired without a breadboard or solder; loose joints caused a floating ADC input, an IMU that dropped off the bus and repeated USB over-current disconnects, and a reversed supply once overheated the IMU. (5) The air-mouse thresholds and angles were validated on synthetic frames and a virtual device; the live-hand demonstration is still to be completed. (6) Only Linux with Hyprland is supported. What I would change: solder every joint and add a strain relief; record a real gesture dataset; add a third flex channel; and fit a magnetometer for a drift-free heading. Next engineering step: measure firmware WCET with a GPIO toggle on a logic analyser, measure supply current, and rerun T1 over 60 s.

References

[1] Espressif Systems, “ESP32 Series Datasheet.”

[2] InvenSense, “MPU-6500 Product Specification,” rev. 1.3.

[3] InvenSense, “MPU-6500 Register Map and Descriptions,” rev. 2.1.

[4] Silicon Labs, “CP2102/9 Single-Chip USB to UART Bridge Data Sheet.”

[5] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” J. ACM, vol. 20, no. 1, pp. 46–61, 1973.

[6] Linux kernel documentation, “Userspace I/O input: uinput,” kernel.org.

[7] G. Tarasov, “python-evdev,” https://python-evdev.readthedocs.io.

[8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[9] S. Ramírez, “FastAPI,” https://fastapi.tiangolo.com.

[10] Hyprland, “Binds (mouse binds and dispatchers),” Hyprland Wiki.

Appendices

A Source code

Repository: https://github.com/dayaj1222/data-glove (branch main). Key files: firmware/glove_full/glove_full.ino; backend/app/aircursor.py, serial_manager.py,

device/framing.py, routers/cursor.py, routers/training.py; frontend/src/components/AirMouseUI.js, SniperUI.js, PuzzleUI.js; frontend/src/lib/pointer.js; tests in backend/tests/ (41 tests).

B Raw data

docs/calibration/gyro_still.jsonl (299 still samples, raw gyro and accelerometer); docs/measurements/results.json (all host timing results, raw latency and release samples); docs/measurements/measure_host.py (script that produced them, run from backend/). Review 2 models and figures are in the separate Review 2 document.

Submission checklist

Item	✓
15–20 pages excluding front matter and appendices; guidance blocks deleted	✓
Every requirement has an ID, a measurable target and a test	✓
At least two alternatives compared for each major decision	✓
Models revised to match the build; changes since Review 2 listed	✓
Traceability table from model elements to code	✓
Optimisation with measured before and after	✓
Repeated measurements with spread; one fault-injection test	✓
Decision log, BOM and contribution table complete	